LAB 3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

0. Members

Student ID	Name
673380416-3	นายพัทธดนย์ คำนัน
673380428-6	นายสิทธิโชค มุขนาค
673380608-4	นางสาวอมลวรรณ พิมพิชัย

1. Objective

วัตถุประสงค์ของ Lab นี้คือ

- ศึกษาการทำงานของ **Router-on-a-Stick (Inter-VLAN Routing)**
- ส่งไฟล์ผ่าน **Custom MIME protocol**
- วิเคราะห์ packet ผ่าน **Wireshark**
- สังเกตการทำงานของ protocol ในแต่ละ OSI Layer ได้แก่

```
Layer 2 : ARP / Ethernet
Layer 3 : IP routing
Layer 4 : TCP
Layer 7 : Application MIME metadata
```

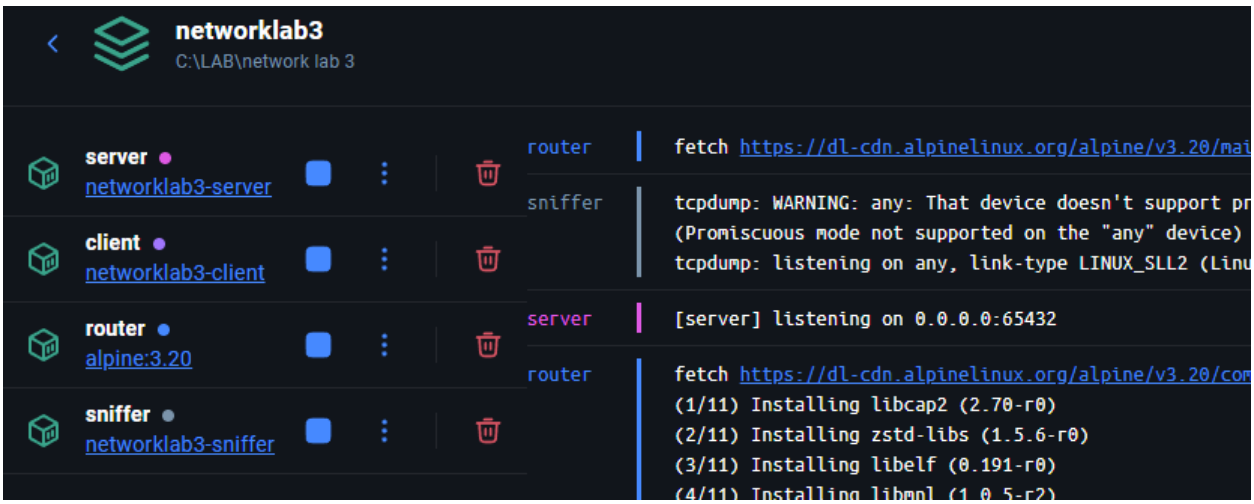
โดยโค้ดของ Lab 4 นักศึกษาได้จัดทำและอัพโหลดขึ้น GitHub เรียบร้อยแล้ว ท่านอาจารย์สามารถดูได้ผ่าน repo ของนักศึกษา ตามลิงค์ที่ได้แนบไว้นี้

GitHub link: https://github.com/ms584/CP352005-Computer-Networks-Laboratory/tree/main/network%20lab%203

2. Network Topology

```
PC1 (Client) VLAN10
192.168.10.10
        |
        |
     Router
  192.168.10.1
  192.168.20.1
        |
        |
PC2 (Server) VLAN20
192.168.20.20
```

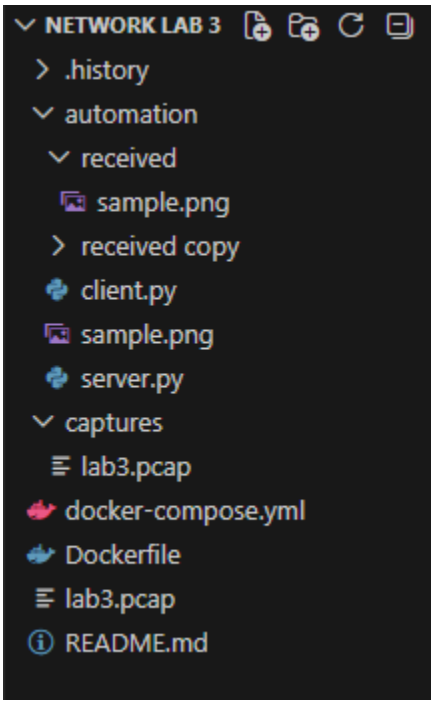
Docker containers



3. Addressing Scheme

Device	Interface	IP
Router	eth0	192.168.10.1
Router	eth1	192.168.20.1
Client	eth0	192.168.10.10
Server	eth0	192.168.20.20

4. Project Structure



5. Docker Network Configuration

Docker Compose ถูกใช้เพื่อจำลอง VLAN network

VLAN10: 192.168.10.0/24

VLAN20: 192.168.20.0/24

Router ทำหน้าที่เป็น gateway

192.168.10.1

192.168.20.1

6. Starting the Environment

เริ่มระบบ: docker compose up --build -d

ตรวจ container: docker ps

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker ps
CONTAINER ID   IMAGE                 COMMAND                  CREATED         STATUS         PORTS     NAMES
3606a4961eee   networklab3-server    "sh -c ' ip route de…"   4 seconds ago   Up 3 seconds             lab3-server
1b17ddb16f0b   networklab3-client    "sh -c 'ip route del…"   4 seconds ago   Up 3 seconds             lab3-client
07ea0dc9b8bd   alpine:3.20           "sh -c 'apk add --no…"   4 seconds ago   Up 3 seconds             lab3-router
578fd420ac10   networklab3-sniffer   "sh -c 'rm -f /captu…"   4 seconds ago   Up 3 seconds             lab3-sniffer
```

7. Clear ARP Cache

เพื่อให้เกิด ARP ใหม่

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec lab3-client ip neigh flush all

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec lab3-server ip neigh flush all

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec lab3-router ip neigh flush all
```

8. Connectivity Test

ทดสอบ network: docker exec -it lab3-client ping -c 10 192.168.20.20

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec -it lab3-client ping -c 10 192.168.20.20
PING 192.168.20.20 (192.168.20.20) 56(84) bytes of data.
64 bytes from 192.168.20.20: icmp_seq=1 ttl=63 time=0.104 ms
64 bytes from 192.168.20.20: icmp_seq=2 ttl=63 time=0.086 ms
64 bytes from 192.168.20.20: icmp_seq=3 ttl=63 time=0.075 ms
64 bytes from 192.168.20.20: icmp_seq=4 ttl=63 time=0.076 ms
64 bytes from 192.168.20.20: icmp_seq=5 ttl=63 time=0.064 ms
64 bytes from 192.168.20.20: icmp_seq=6 ttl=63 time=0.059 ms
64 bytes from 192.168.20.20: icmp_seq=7 ttl=63 time=0.057 ms
64 bytes from 192.168.20.20: icmp_seq=8 ttl=63 time=0.061 ms
64 bytes from 192.168.20.20: icmp_seq=9 ttl=63 time=0.062 ms
64 bytes from 192.168.20.20: icmp_seq=10 ttl=63 time=0.061 ms

--- 192.168.20.20 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9205ms
rtt min/avg/max/mdev = 0.057/0.070/0.104/0.014 ms
```

แสดงว่า **inter-VLAN routing** ทำงาน

9. Start Packet Capture

เริ่ม capture packet จาก router: docker exec -it lab3-router sh

tcpdump -i any -w /tmp/lab3.pcap arp or tcp port 65432

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec -it lab3-router sh
/ # tcpdump -i any -w /tmp/lab3.pcap arp or tcp port 65432
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C71 packets captured
```

10. Generate ARP Traffic

docker exec -it lab3-client ping -c 1 192.168.20.20

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec -it lab3-client ping -c 10 192.168.20.20
PING 192.168.20.20 (192.168.20.20) 56(84) bytes of data.
64 bytes from 192.168.20.20: icmp_seq=1 ttl=63 time=0.104 ms
64 bytes from 192.168.20.20: icmp_seq=2 ttl=63 time=0.086 ms
64 bytes from 192.168.20.20: icmp_seq=3 ttl=63 time=0.075 ms
64 bytes from 192.168.20.20: icmp_seq=4 ttl=63 time=0.076 ms
64 bytes from 192.168.20.20: icmp_seq=5 ttl=63 time=0.064 ms
64 bytes from 192.168.20.20: icmp_seq=6 ttl=63 time=0.059 ms
64 bytes from 192.168.20.20: icmp_seq=7 ttl=63 time=0.057 ms
64 bytes from 192.168.20.20: icmp_seq=8 ttl=63 time=0.061 ms
64 bytes from 192.168.20.20: icmp_seq=9 ttl=63 time=0.062 ms
64 bytes from 192.168.20.20: icmp_seq=10 ttl=63 time=0.061 ms

--- 192.168.20.20 ping statistics ---
10 packets transmitted, 10 received, 0% packet loss, time 9205ms
rtt min/avg/max/mdev = 0.057/0.070/0.104/0.014 ms
```

จะเกิด ARP request, ARP reply

11. MIME File Transfer

ส่งไฟล์จาก client ไป server

docker exec -it lab3-client sh -c "python /app/client.py /app/sample.png --host 192.168.20.20 --port 65432"

ผลลัพธ์

```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec -it lab3-client sh -c "python /app/client.py /app/sample.png --host 192.168.20.20 --port 65432"
[client] sent header:
{
  "mime_type": "image/png",
  "size": 156,
  "filename": "sample.png"
}
[client] server response:
{"status": "ok", "saved_as": "/app/received/sample.png", "received_size": 156, "declared_mime": "image/png", "guessed_mime": "image/png"}
```

12. Stop Capture

หยุด tcpdump: CTRL + C

copy pcap: docker cp lab3-router:/tmp/lab3.pcap .

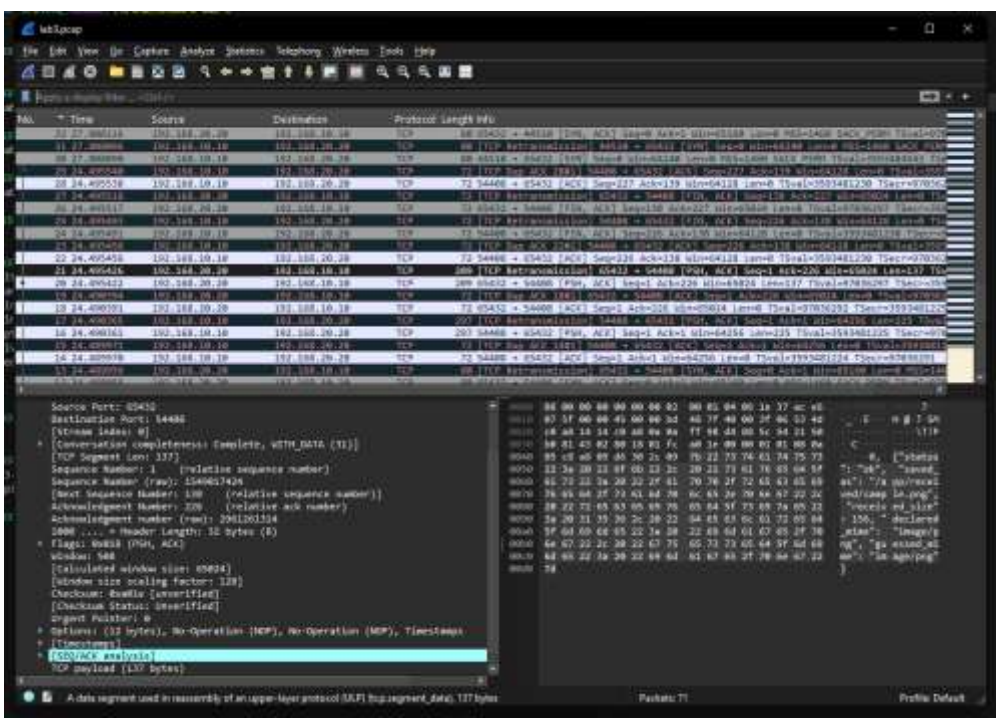
```
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker exec -it lab3-router sh
/ # tcpdump -i any -w /tmp/lab3.pcap arp or tcp port 65432
tcpdump: data link type LINUX_SLL2
tcpdump: listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
^C71 packets captured
71 packets received by filter
0 packets dropped by kernel
/ #
/ #
/ #
Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$

Xytron404@DESKTOP-1FBNV5Q MINGW64 /c/LAB/network lab 3
$ docker cp lab3-router:/tmp/lab3.pcap .
Successfully copied 10.2kB to C:\LAB\network lab 3\.
```

13. Wireshark Analysis

เปิดไฟล์

lab3.pcap

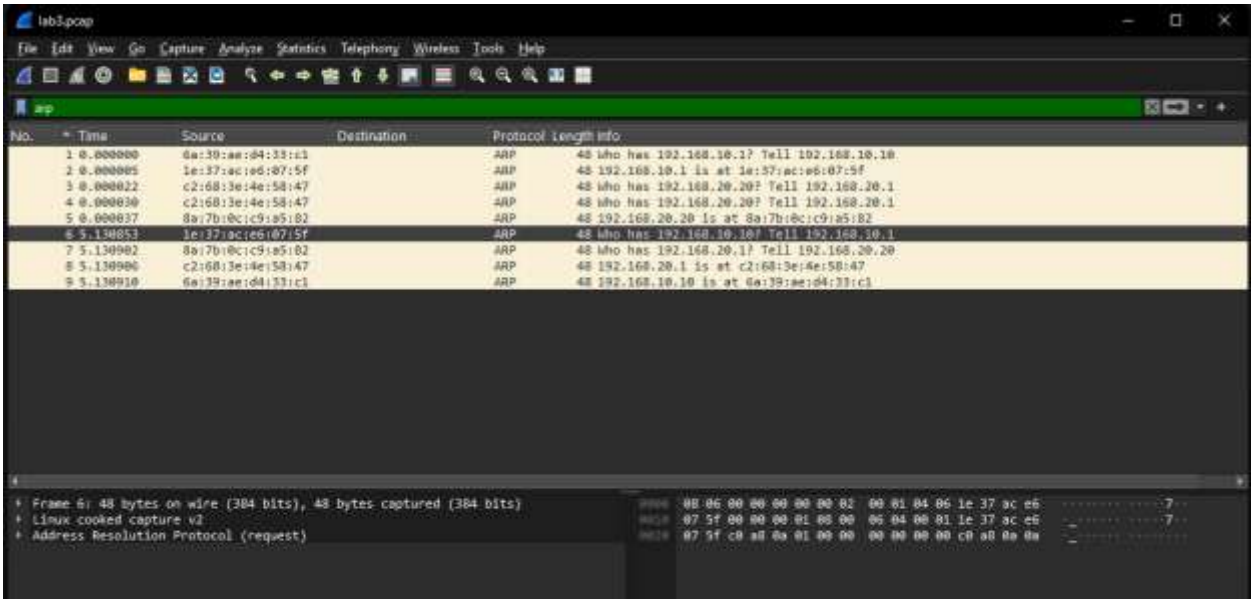


14. ARP Analysis

Filter

arp

ผลที่พบ



Who has 192.168.10.1? Tell 192.168.10.10

192.168.10.1 is at xx:xx:xx:xx

Who has 192.168.20.20? Tell 192.168.20.1

192.168.20.20 is at xx:xx:xx:xx

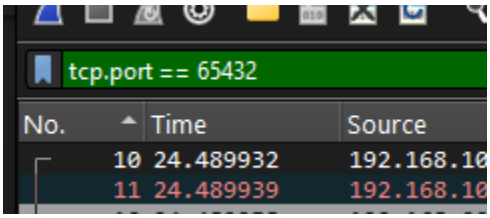
แสดงว่า

- client หา router
- router หา server

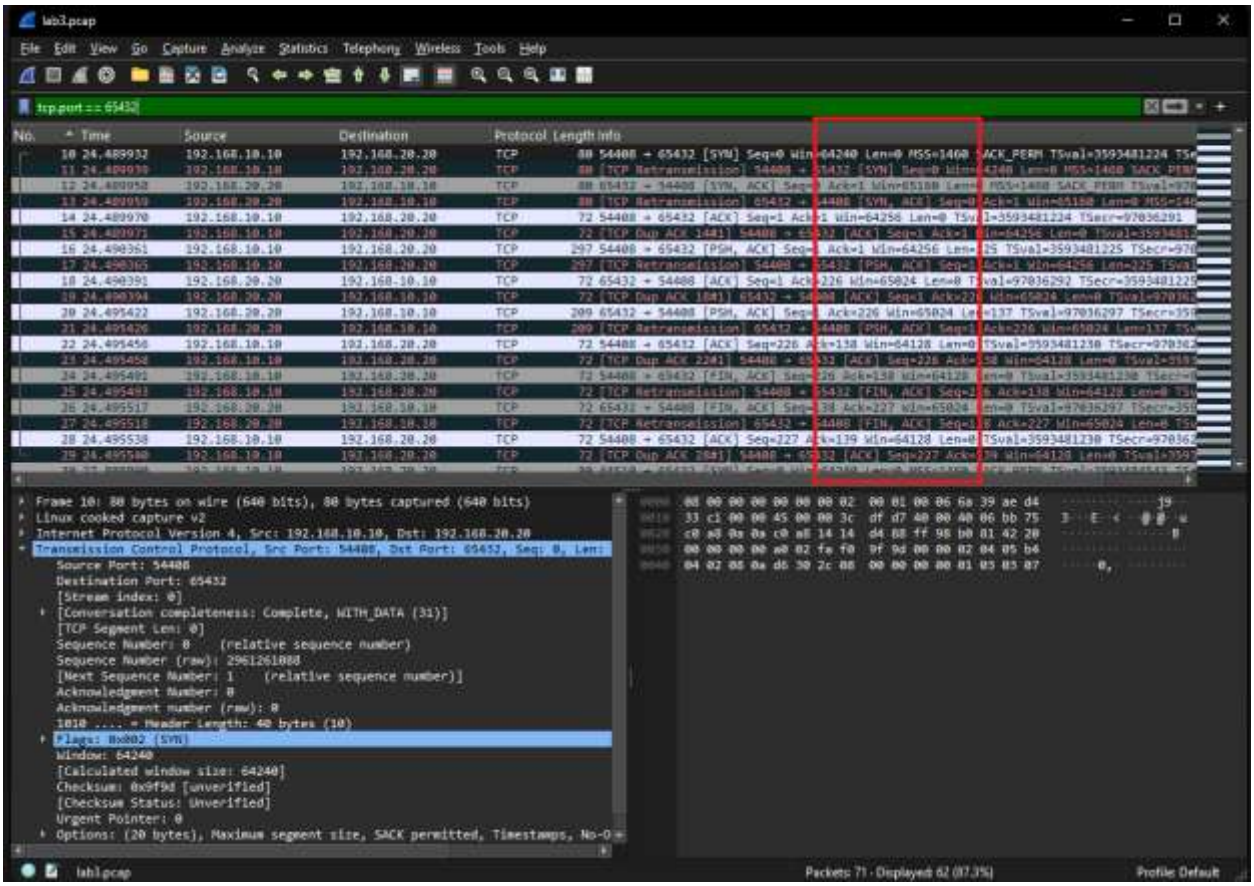
15. TCP Three-Way Handshake

Filter

tcp.port == 65432



พบ sequence



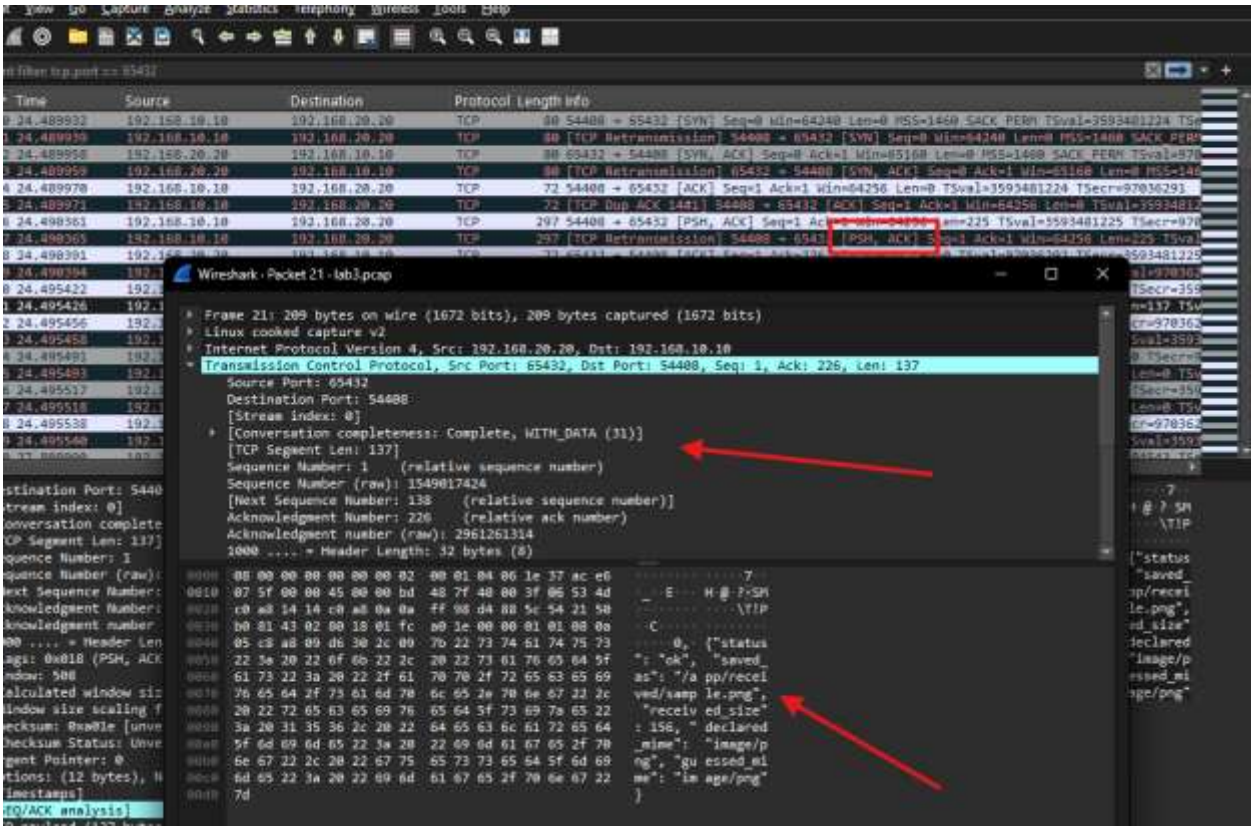
SYN

SYN-ACK

ACK

แสดงการสร้าง TCP connection

16. Data Transfer



Packet type

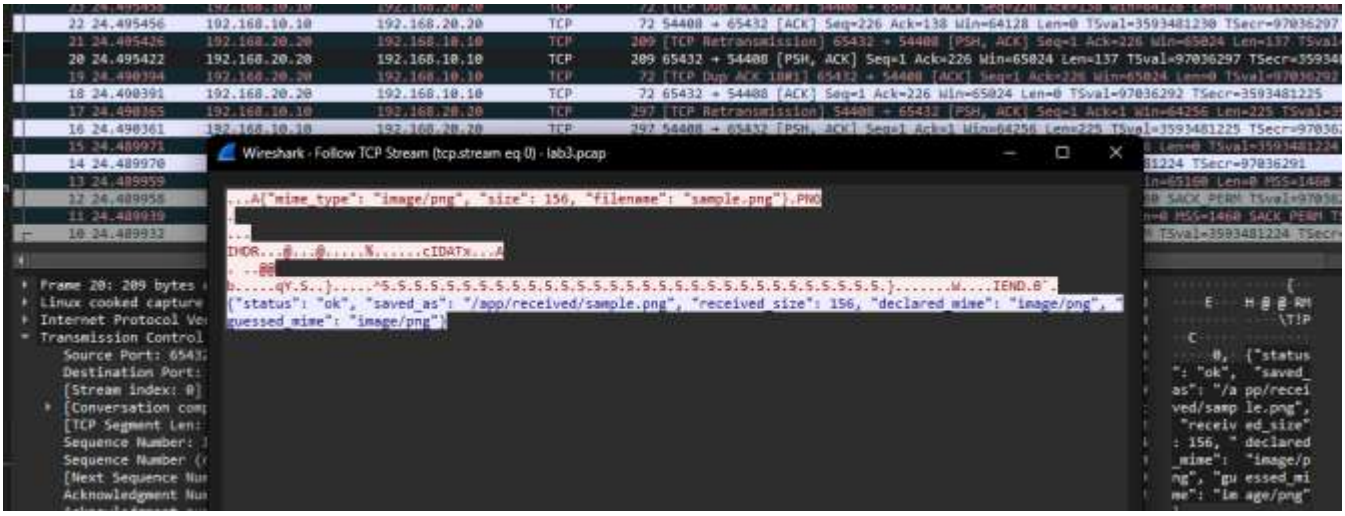
PSH ACK

แสดงการส่ง data

17. MIME Header Inspection

คลิก packet, Right Click, Follow → TCP Stream

พบข้อมูล



{

"mime_type": "image/png",

"size": 156,

"filename": "sample.png"

}

ตามด้วย binary PNG

18. Server Response

```
...A{"mime_type": "image/png", "size": 156, "filename": "sample.png"}.PNG
.
...
IHDR...@...@.....%......cIDATx...A
. ..@@
b.....qY.S..}.....^5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.5.}.......W....IEND.B`.
{"status": "ok", "saved_as": "/app/received/sample.png", "received_size": 156, "declared_mime": "image/png", "
guessed_mime": "image/png"}
```

หลังรับไฟล์ server ส่ง response

{

"status": "ok",

"saved_as": "/app/received/sample.png",

"received_size": 156,

"declared_mime": "image/png",

"guessed_mime": "image/png"

}

19. OSI Layer Observation

Layer	Protocol	Observation
L2	ARP	MAC address resolution
L3	IP	routing between VLAN
L4	TCP	reliable connection
L7	MIME	file metadata

20. Packet Flow

ลำดับ packet

```
ARP Request
ARP Reply

TCP SYN
TCP SYN ACK
TCP ACK

JSON MIME header
PNG binary data

Server response

TCP FIN
```

21. MIME Protocol Structure

Packet format

```
[4 bytes header length]
[JSON header]
[file binary payload]
```

Example

{

"mime_type": "image/png",

"size": 156,

"filename": "sample.png"

}

---

22. Key Wireshark Filters

ARP: arp

TCP: tcp.port == 65432

Client traffic: ip.addr == 192.168.10.10

Data packets: tcp.len > 0

23. Conclusion

Lab นี้แสดงให้เห็นการทำงานของ network stack ตั้งแต่

- Ethernet → ARP → IP → TCP → Application
- Router-on-a-Stick ทำให้ VLAN สามารถสื่อสารกันได้ผ่าน Layer 3 routing
- Custom MIME protocol ถูกใช้ในการส่ง metadata และ binary file ผ่าน TCP connection
- Wireshark ทำให้สามารถวิเคราะห์ packet ได้ละเอียดถึงระดับ byte

24. Learning Outcome

หลังจากทำ lab นี้สามารถ

- เข้าใจ inter-VLAN routing
- วิเคราะห์ ARP และ TCP handshake
- ใช้ Wireshark วิเคราะห์ packet
- สร้าง protocol สำหรับส่งไฟล์
- เข้าใจการ encapsulation ของ network

Assessment Questions — Lab 3

1. Why does TCP retransmit lost segments?

ANSWER TCP retransmits lost segments because TCP is a reliable transport protocol. When a sender transmits a TCP segment, it expects an acknowledgment (ACK) from the receiver. If the sender does not receive the ACK within a specific timeout period, or if duplicate ACKs are detected, TCP assumes the segment was lost and retransmits it.

Retransmission ensures that:

- all data arrives correctly
- packets lost due to congestion or errors are resent
- the data stream remains reliable

In the Wireshark capture of this lab, retransmissions were observed as:

[TCP Retransmission]

These occurred when the sender did not immediately receive an ACK from the receiver.

2. How does VLAN tagging affect broadcast domains?

ANSWER VLAN tagging separates a physical network into multiple logical broadcast domains.

Each VLAN acts as an independent Layer-2 network. Broadcast traffic (such as ARP requests) is only delivered to devices within the same VLAN.

For example in this lab:

```
VLAN10 → 192.168.10.0/24
VLAN20 → 192.168.20.0/24
```

Devices in VLAN10 cannot directly broadcast to VLAN20.
Therefore:

- ARP broadcast from 192.168.10.10 stays inside VLAN10
- Router performs Layer-3 routing to reach VLAN20

This segmentation improves:

- network security
- traffic isolation
- scalability

3. What role does MIME play compared to TCP headers?

ANSWER TCP headers handle transport-level communication, while MIME handles application-level metadata.

TCP provides:

- reliability
- ordering
- flow control
- retransmission

MIME provides information about the content being transferred, such as:

```
{
 "mime_type": "image/png",
 "size": 156,
 "filename": "sample.png"
}
```

In this lab:

- TCP ensured reliable delivery
- MIME described the file type and metadata

Thus:

TCP = transport mechanism

MIME = application data description

4. Where does routing decision occur?

<u>ANSWER</u> Routing decisions occur at Layer 3 (Network Layer) on the router.

In this lab, the router container has two interfaces:

192.168.10.1

192.168.20.1

When the client sends traffic to:

192.168.20.20

the process is:

1. Client sends packet to its gateway

   192.168.10.1

2. Router examines destination IP

   192.168.20.20

3. Router forwards the packet through interface connected to VLAN20.

This is called inter-VLAN routing.

5. Why does ARP precede TCP communication?

<u>ANSWER</u> Before TCP communication can occur, the sender must know the MAC address of the destination device or gateway.

ARP (Address Resolution Protocol) resolves:

IP address → MAC address

Example from the capture:

Who has 192.168.10.1? Tell 192.168.10.10

192.168.10.1 is at xx:xx:xx:xx

Once the MAC address is known:

1. Ethernet frame can be built
2. IP packet can be encapsulated
3. TCP connection can start

Therefore the sequence is:

ARP → TCP handshake → Data transfer

Reflection Exercise

How does application meaning survive multiple encapsulation layers?

<u>ANSWER</u> Application data (such as MIME metadata) is encapsulated through multiple protocol layers:

```
Application
↓
TCP
↓
IP
↓
Ethernet
```

Even though the data is wrapped inside several headers, the original application content remains intact because each layer only processes its own header information.

When the packet reaches the destination, the layers are removed in reverse order:

Ethernet → IP → TCP → Application

This process ensures that the original data, including MIME metadata and binary payload, is preserved during transmission.